SSA Star/Streak Annotation: Written Response

AI/ML Data Annotation Intern Assessment, Digantara

Approach. I used a scripted, threshold-based pipeline (`annotate.py`, see README) instead of a UI tool: the frames are 61 Mpx single-band images with hundreds to thousands of sources each, and a star is only a few pixels wide, so hand-tracing pixel-accurate outlines is not practical. The script builds a pixel-level mask for every object, labels it *star* (class 0) or *streak* (class 1), cuts 1024 × 1024 tiles and writes YOLO-Ultralytics segmentation labels. The output is **automatic candidate detection**, not verified ground truth: there is no ground truth for these frames, and the review status is stated at the end.

a) Pre-processing steps applied

1. **Read as 16-bit unsigned.** The FITS has `BITPIX=16`, `BZERO=32768`; astropy returns `uint16`. Nothing is rescaled or clipped before measuring. (Such files cannot be memory-mapped, so `memmap=False`.)
2. **Background and noise mesh.** 128 px blocks; in each block the sigma-clipped *mean* and *standard deviation* (4 rounds of 3σ clipping, so stars and streaks do not bias them). The mesh is median-filtered (3 × 3) and bilinearly upsampled. This removes sky gradients and vignetting and gives a local noise level, so one threshold in SNR units works across a frame and across frames. **Why not median/MAD:** I first used it, and the real frames broke it. They are low-count integer images (typical pixel value 1–3 counts, ~100 distinct values in a 2048^2 crop), so more than half the pixels in a block share one value and the MAD is 0: the diagnostic printed mesh noise 0.00 against 1.05 from neighbour differences. Result: 0 detections on one frame, and 15,525 stars plus 998 streaks on another. A noise floor (quantisation noise 0.29, and half the typical noise) stops dead, masked regions from turning one stray count into a huge SNR.
3. **Matched filter.** Gaussian smoothing ($\sigma = 1$ px, about the PSF scale) of the background-subtracted frame; the noise reduction of the filter is measured on the frame itself and used to convert the result to a per-pixel **SNR map**.
4. **Hysteresis threshold.** Seeds need SNR ≥ 6, the mask grows to neighbours with SNR ≥ 2.5, so the polygon follows the object's true extent and bridges gaps along faint streaks. The seed level was 5 at first; on the 8-bit reference tile (early version of the code, rough count against the provided annotation) it gave ~61 blobs against ~31 annotated, and 6 gives 41.
5. **Artefact rejection.** (i) Components under 4 px are dropped. (ii) *Hot pixels / cosmic rays:* a real source has a PSF spanning several pixels, so the 8 neighbours of its brightest pixel carry signal; if they carry less than 2.5σ in total the component is dropped. My first rule ("peak holds ≥ 60% of the 3 × 3 flux") deleted real, sharp stars on the real frames (1,025 of 2,358 components in one crop, peak SNR up to 201); the neighbour test replaced it. (iii) Long components hugging the frame border (hot rows/columns) are dropped.

6. **Display stretch (tiles and overlays only).** asinh of the background-subtracted frame in units of local noise, one fixed scale per frame, for the 8-bit tiles. Detection never uses the 8-bit data; lossless 16-bit tiles are saved alongside.

Assumptions: single exposures, no dark/flat frames, so no calibration beyond the mesh background; PSF about 2–4 px FWHM; saturation at 65535.

b) Boundaries: 9568 × 6380 into exact 1024 × 1024 tiles

9568 = 9 × 1024 + 352 and 6380 = 6 × 1024 + 236, so the frame is not a multiple of the tile size. I use a ceil grid of **10 columns × 7 rows = 70 tiles** and pad the right edge by 672 px and the bottom by 788 px (canvas 10240 × 7168).

- **Reflect padding, not zeros.** A black band lies far below the noise floor and is a trivially learnable edge; reflection keeps noise and background continuous. Padded pixels never carry labels (checked by `validate_dataset.py`).
- **No cropping, no resizing:** every source pixel is in exactly one tile (unit-tested: coverage count is 1 everywhere, stitched tiles equal the source). Grid origin (0, 0); tiles are named `<frame>_r<row>_c<col>`; `*_meta.json` stores original and padded size.
- **Detect on the whole frame, then cut the label map.** Streaks are long; a per-tile detector would split one into pieces that each look like short blobs and mis-measure length and aspect. An object crossing a tile border appears as a clipped polygon in each tile it touches (slivers under 3 px dropped). Such split streaks are a known limitation for training: the same object is two separate polygons.
- **Repatching** puts each tile back at its offset and crops to 9568 × 6380. **On all 10 real frames** the repatched 16-bit mosaic is bit-identical to the FITS data and the 8-bit mosaic identical to the stretched frame (`annotate.py repatch --fits-dir`, all checks `True`); the same holds on synthetic frames.
- **Polygon validity.** `validate_dataset.py` checks every label (class id, coordinates in [0, 1], ≥ 3 distinct vertices, non-zero area, no self-crossing, no labels in padding, image/label pairing). It found self-crossing polygons on my first export: contour simplification made 1-px spurs on star outlines cross themselves. The exporter now tests each polygon after rounding and falls back (simplified → finer → raw contour → coarser simplifications up to 3 px, which keep concavities → convex hull → oriented box). Holes inside objects are ignored (outer contour only).
- **Masks and manifest.** `annotate.py` writes exact pixel-level masks, `dataset/masks/<tile>.png` (1024 × 1024 uint8: 0 background, 1 star, 2 streak), straight from the detection label map. Each pixel carries one label, so classes never overlap, and padding is always 0; `make_masks.py` checks that every mask is aligned with its image tile. The YOLO polygons are the simplified outer contour of the same objects (holes ignored, tiny slivers dropped); rasterised, they agree with the exact masks at foreground IoU 0.99 on the synthetic frame (not measured on the real frames). `dataset/manifest.csv` has one row per tile: source frame, row/col, pixel box in the source frame, padding, polygon counts and file paths.

c) Short/fat streaks vs blobs

Every connected component is measured with image moments (major/minor axis `L`, `w` of the equivalent ellipse). It is a **streak** if any rule holds:

- **Mask rule:** `L ≥ 14 px` and `L/w ≥ 3.0`; or `L ≥ 40 px` and `L/w ≥ 2.0` (bent or noisy long streaks measure less elongated).
- **Core rule (short, fat streaks):** the same measurement on the *core* only (pixels above 30% of the object's own peak SNR, not below 2.5) must give `L_core ≥ 7 px` and `L_core/w_core ≥ 1.9`, **and peak SNR ≥ 9**.
- **Fragmented faint streaks:** fragments within 3 px are grouped; if the group is collinear (`L ≥ 30 px`, aspect ≥ 4.5) they are merged and the gaps closed, giving one polygon instead of several star-like pieces.

Why the core rule: with the mask rule alone, 15 of 56 detected simulated streaks were labelled stars (bright 12–24 px streaks whose wings widen the mask, aspect 2.2–3.0). A star's core stays round at any brightness; a short streak's core stays elongated. *Why the SNR ≥ 9 condition:* on the reference image the first version labelled 10 objects as streaks against 2 annotated; the 8 extras were faint clumps (peak SNR ≤ 7, ≤ 12 px) whose shape is set by noise. With the condition, 2 streaks are found (the thin one at upper left and the thick one at lower right; I matched them to the reference by eye only).

Saturated and defocused stars are large but round (aspect ≈ 1) and stay stars. **Ambiguous cases** are those near the thresholds (aspect 1.9–3, `L` 7–14 px, a star touching a streak, close doubles). Each object's `length`, `aspect`, `core_*` and `peak_snr` are in `*_meta.json`, so they can be sorted and reviewed first; the labels import into CVAT for correction. A streak seen end-on, shorter than ~7 px, cannot be told from a star and is labelled star.

d) Faint and small blobs: the star class

- **Included:** any non-rejected component with peak smoothed SNR ≥ 6, area ≥ 4 px that does not meet a streak rule, including saturated stars. The mask extends to the 2.5σ contour, so the polygon covers the PSF core and inner halo; a bright star's halo belongs to its polygon and is not a separate object.
- **Faint end:** SNR 6 on the matched-filtered image is the cut-off, judged against local noise (mesh σ), not a fixed count, so it is consistent across the vignetting. Below it, noise peaks start to outnumber real sources and annotating them adds label noise; I prefer omitting sub-threshold sources to noisy labels, and this is a limitation. The filter output is correlated, so the Gaussian tail rate is not a valid false-alarm estimate; I measured false positives on simulated noise instead (below).
- **Excluded:** single hot pixels, cosmic rays, specks under 4 px. Tiny blobs whose contour has fewer than 3 vertices get a small oriented box (1 px margin) so the YOLO label stays valid.
- **Uncertain detections** (peak SNR 6–7, area near 4 px) are the first to check in review.

Validation and results

Simulated frame with ground truth (3500 × 2300 uint16, 940 stars incl. saturated and defocused ones, 60 streaks of 12–140 px, 300 hot pixels; about 117 stars per Mpx, comparable to the 52–293 stars per Mpx detected on the real frames; current code, `evaluate_synthetic.py`):

Check	seed SNR 6 (default)	seed SNR 5
Star recall	90.5% (851/940)	93.7% (881/940)
Streak recall	81.7% (49/60); 9 of 11 misses < 150 counts/px	85.0% (51/60)
Unmatched detections	2 of 868	4 of 902
Stars labelled streak	16: 10 merged with a streak they touch, 6 are 3 close doubles	16
Streaks labelled star	5 (short ones)	6

The remaining check items (tile coverage, lossless repatch, label validity, hot-pixel rejection rate) are in `tests/` and `validate_dataset.py`. Not measured: precision and recall on the real frames, since they have no ground truth. Real data is harder than the simulation (non-Gaussian noise, vignetting, masked regions).

Real frames. They show strong vignetting and large dark masked regions with stair-step edges. With the final code and seed 6, all 10 frames were processed (14–46 s per frame, as printed by the run); 700 tiles and label files passed `validate_dataset.py`. Automatic counts (stars / streaks) for five example frames: 1a600998: 3,203 / 64; 3e6beceb: 9,809 / 220; 4a8e6cd4: 5,896 / 126; 4e2cdd92: 5,959 / 112; 66ed4268: 17,889 / 639. All 10 frames are in `outputs/object_counts.md` and each `*_meta.json`. **Review status:** these are automatic detections. I have not verified them object by object; I viewed a downscaled overlay of one frame (4e2cdd92), where blue marks followed the real star distribution and the clearest red marks sat on short elongated objects, with a few tiny red specks that may be faint clumps. The 639 streaks on the busiest frame probably include noise clumps and dark-region edges, which I have not confirmed. Next steps: review the overlays (largest `length` and lowest `peak_snr` first), raise the streak SNR/length limits if clumps dominate, correct in CVAT.